

## Overview

This essay explains the algorithmic design behind a GPU pipeline that converts a UTF8 byte stream into UTF32 code points and then compacts those code points into contiguous output using a parallel, strided doubling merge. The focus is on the why and how of the algorithm: reservation and counting during expansion, padding to align work groups, leader/partner merging for compaction, and the correctness and performance tradeoffs that follow.

---

### Two distinct phases and two lengths

The pipeline separates work into two conceptual phases:

- Expansion: read the UTF8 input bytes and produce Unicode code points (UTF32 tokens). The number of tokens produced is not necessarily equal to the number of input bytes, so the algorithm must discover the expanded length at runtime.
- Compaction: take the produced UTF32 tokens and pack them into contiguous runs, merging adjacent segments in parallel until a final compacted sequence is produced.

Because these phases operate on different granularities, the algorithm tracks two lengths: the original UTF8 byte count (input length) and the expanded UTF32 token count. Both are needed to drive iteration bounds, padding, and final results.

---

### Expansion by reservation and local writes

The expansion stage uses a reservation pattern rather than global synchronization:

- Each decoding worker processes a subset of input bytes. When it needs to emit one or more UTF32 tokens, it reserves destination slots in the expanded buffer via an atomic counter. Reservation guarantees unique, nonoverlapping write positions without a global barrier.
- After writing tokens into the reserved slots, the worker marks those slots as ready (a perslot flag). The atomic counter also serves as the authoritative count of emitted tokens, so the expanded length becomes available to other stages without a separate reduction step.
- This approach supports variable expansion rates (e.g., multibyte sequences, combining characters) and avoids expensive global coordination during decoding.

---

### Padding to align work groups

To make the compaction algorithm uniform and branchminimal, the expanded token stream is padded up to a multiple of the GPU workgroup size:

- Padding slots contain a sentinel token and are assigned zero logical length. They are treated as ready so they do not block progress.
- Padding eliminates the special cases that arise when the number of tokens is odd or not divisible by the grouping used by the merge algorithm. With padding, every merge round has complete leader/partner pairs, which simplifies the control flow and reduces divergent branches across threads.

---

## Stride doubling compaction with leader and partner

Compaction uses a stridedoubling merge that repeatedly combines adjacent segments in parallel:

- Initialization: each index is considered a segment of length 1 (or 0 for padding) with a position pointer. A perslot status flag indicates readiness.
- Merge rounds: for stride values  $(1, 2, 4, \dots)$ , a thread becomes a leader for indices where the index modulo  $(2 \cdot \text{stride})$  equals the stride. Each leader waits until its left partner is ready, then appends the partner's segment to its own region and updates its length and readiness flag.
- This pattern halves the number of active segments each round and completes in  $(O(\log N))$  rounds. The leaderanchored approach keeps writes localized to the leader's region, avoiding complex pointer shuffling.

---

## Correctness, metrics, and synchronization choices

Correctness is enforced by a combination of perslot readiness and atomic counters:

- Perslot status flags let leaders spinwait for partners without global barriers. This localized waiting is sufficient because each merge depends only on two slots.
- Atomic operations serve two roles: reservation during expansion and global metrics during compaction. The algorithm updates two global counters: one that records the maximum merged segment length (the final UTF32 length) and another that accumulates the number of tokens written. These counters let the host read final results without additional reductions.
- The design intentionally avoids global barriers and memory fences between stages; instead it relies on the readiness flags and atomics to order dependent operations. This reduces synchronization overhead but requires careful handling of spin waits and memory visibility semantics.

---

## Performance tradeoffs and practical considerations

The algorithm balances throughput, memory locality, and simplicity:

- Pros: Reservation avoids contention for write positions; padding removes branch divergence; stridedoubling gives logarithmic rounds; atomics provide lightweight global metrics.
- Cons: Spinwaiting can waste cycles if producers are slow; many small atomic operations can limit throughput on highly contended hardware; copying partner segments into leader regions can cause noncoalesced memory writes if

layout is not tuned.

- Tuning knobs: choose group size to match hardware, size buffers to allow coalesced writes, and consider batching or hierarchical merges to reduce atomic pressure. For variablelength expansion, reserving larger contiguous blocks per producer can improve locality at the cost of more complex reservation logic.

---

## Conclusion

Algorithmically, the pipeline is a pragmatic combination of lockfree reservation, padding for uniformity, and parallel, leaderdriven stridedoubling merges. It cleanly separates decoding from compaction, preserves the original API semantics (input length as UTF8 bytes), and produces reliable global metrics via atomics. The result is a scalable, GPUfriendly approach that trades a small amount of extra memory and atomic work for a simple, branchminimal compaction algorithm that performs well on massively parallel hardware.